

UNIT-3

REACT FOR FRONT-END DEVELOPMENT

3.1 LISTS AND KEYS

3.1.1 Lists

Lists are an important aspect of your app. Every application is bound to make use of lists in some form or the other. You could have a list of tasks like a calendar app, a list of pictures like Instagram, a list of items to shop in a shopping cart, and so on. The use cases are numerous. Lists within an application can be performance-heavy. Imagine an app with a huge list of videos or pictures and you keep getting thousands more, as you scroll. That could take a toll on the app's performance.

Because performance is an important aspect, when you use lists, you need to ensure they are designed for optimal performance.

In JavaScript, the `.map()` method iterates through the parent array, calling a function on each element. Then it creates a new array containing the values that have been changed. It does not affect the parent array.

Lists are very useful when it comes to developing the UI of any website. Lists are mainly used for displaying menus on a website, for example, the navbar menu. In regular JavaScript, we can use arrays for creating lists.

Creating and traversing a list:

We can similarly create lists in React as we do in regular JavaScript i.e. by storing the list in an array. To traverse a list we will use the `map()` function.

Step 1: Create a list of elements in React in the form of an array and store it in a variable. We will render this list as an unordered list element in the browser.

Step 2: We will then traverse the list using the JavaScript `map()` function and update elements to be enclosed between `<li></li>` elements.

Step 3: Finally, we will wrap this new list within `<ul></ul>` elements and render it to the DOM.

App.js

```
import React from 'react';
import ReactDOM from 'react-dom'
function App()
{
    const numbers = [1,2,3,4,5];
    const updatedNums = numbers.map((number)=>{ return <li>{number}</li>;});
```

```
ReactDOM.render(  
  <ul>  
    {updatedNums}  
  </ul>,  
  document.getElementById('root')  
)  
}  
export default App;
```

Output: The above code will render an unordered list as shown below

- 1
- 2
- 3
- 4
- 5

Example

NameList.js

```
import React from 'react';  
import ReactDOM from 'react-dom';  
function NameList(props)  
{  
  const listItems = myList.map((item) => <li>{item}</li> );  
  return (  
    <ul>{listItems}</ul>  
  );  
}  
const myList = ["apple", "orange", "strawberry", "blueberry", "avocado"];  
ReactDOM.render(  
  <NameList myList={myList} />,  
  document.getElementById('root')  
)  
export default NameList;
```

App.js

```
import React from 'react'
import NameList from './components/NameList';
function App()
{
    return (
        <div className='App'>
            <NameList></NameList>
        </div>
    );
}
export default App;
```

Output:

- apple
- orange
- strawberry
- avocado

3.1.2 Keys

What is a key in React?

A “key” is a special string attribute you need to include when creating lists of elements in React. Keys are used in React to identify which items in the list are changed, updated, or deleted.

Keys are used to give an identity to the elements in the lists. It is recommended to use a string as a key that uniquely identifies the items in the list. Assigning keys to the list.

You can assign the array indexes as keys to the list items. The below example assigns array indexes as keys to the elements.

The element's key is a specific property that must be included in the element, and it must be a string. Each list's key should be distinct, which means you shouldn't use values that are identical to the key. Each item within an array must have a unique key, but the key need not be globally unique. The same key can be used across various unrelated components and lists. To put it differently, keys should be unique among siblings rather than globally.

The element's key serves as a form of identifier for React, allowing it to figure out which element was updated, added, or removed.

It's a good idea to pick a value that serves as a unique identifier for each item in the array, which is usually the ID.

Syntax:

```
const numbers = [1, 2, 3, 4, 5];
const updatedNums = numbers.map((number, index) =>
  <li key={index}>
    {number}
  </li>
);
```

An issue with assigning index as keys:

Assigning indexes as keys are highly discouraged because if the elements of the arrays get reordered in the future, then it will get confusing for the developer as the keys for the elements will also change.

Difference between keys and props in React:

Keys are not the same as props, only the method of assigning a “key” to a component is the same as that of props. Keys are internal to React and cannot be accessed from inside of the component like props. Therefore, we can use the same value we have assigned to the Key for any other prop we are passing to the Component.

Using Keys with Components:

Consider a situation where you have created a separate component for list items and you are extracting list items from that component. In that case, you will have to assign keys to the component you are returning from the iterator and not to the list items. That is you should assign keys to `<Component />` and not to `<li>`. A good practice to avoid mistakes is to keep in mind that anything you are returning from inside of the `map()` function is needed to be assigned a key.

Example

```
import React from 'react';
import ReactDOM from 'react-dom';
function ListComponent(props)
{
  const listItems = myList.map((item) =>
    <li key={item.id}>
      {item.value}
    </li>
  );
};
```

```

return (
  <ol type='A'>{listItems}</ol>
);
}
const myList = [{id: 'a', value: "Lotus"},
  {id: 'b', value: "Rose"},
  {id: 'c', value: "Sunflower"},
  {id: 'd', value: "Marigold"},
  {id: 'e', value: "Lily"}];
ReactDOM.render(
  <ListComponent myList={myList} />,
  document.getElementById('root')
);
export default ListComponent

```

Output:

- A. Lotus
- B. Rose
- C. Sunflower
- D. Marigold
- E. Lily

3.2 React Lifecycle**Lifecycle of Components**

Each component in React has a lifecycle which you can monitor and manipulate during its three main phases.

The three phases are: **Mounting, Updating, and Unmounting.**

3.2.1 Mounting

Mounting means putting elements into the DOM.

React has four built-in methods that get called, in this order, when mounting a component:

- constructor()
- getDerivedStateFromProps()
- render()
- componentDidMount()

The `render()` method is required and will always be called, the others are optional and will be called if you define them.

3.2.1.1 constructor

The `constructor()` method is called before anything else, when the component is initiated, and it is the natural place to set up the initial state and other initial values.

The `constructor()` method is called with the props, as arguments, and you should always start by calling the `super(props)` before anything else, this will initiate the parent's constructor method and allow the component to inherit methods from its parent (`React.Component`).

3.2.1.2 `getDerivedStateFromProps`

The `getDerivedStateFromProps()` method is called right before rendering the element(s) in the DOM.

This is the natural place to set the state object based on the initial props.

It takes the state as an argument and returns an object with changes to the state.

The example below starts with the favourite color being "red", but the `getDerivedStateFromProps()` method updates the favourite color based on the `favcol` attribute:

```
ReactDOM.render(<Header favcol="yellow"/>, document.getElementById('root'));
```

Render:

The `render()` method is required and is the method that outputs the HTML to the DOM.

3.2.1.3 `componentDidMount`

The `componentDidMount()` method is called after the component is rendered.

This is where you run statements that require that the component is already placed in the DOM.

3.2.2 Updating

The next phase in the lifecycle is when a component is updated.

A component is updated whenever there is a change in the component's state or props.

React has five built-in methods that get called, in this order, when a component is updated:

`getDerivedStateFromProps()`

`shouldComponentUpdate()`

`getSnapshotBeforeUpdate()`

`render()`

`componentDidUpdate()`

The `render()` method is required and will always be called, the others are optional and will be called if you define them.

3.2.2.1 `getDerivedStateFromProps`

Also, at updates, the `getDerivedStateFromProps` method is called. This is the first method that is called when a component gets updated.

This is still the natural place to set the state object based on the initial props.

Imagine an example has a button that changes the favourite color to blue, but since the `getDerivedStateFromProps()` method is called, which updates the state with the color from the `favcol` attribute, the favourite color is still rendered as yellow.

3.2.2.2 `shouldComponentUpdate`

In the `shouldComponentUpdate()` method you can return a Boolean value that specifies whether React should continue with the rendering or not.

The default value is `true`.

3.2.2.3 `getSnapshotBeforeUpdate`

The `getSnapshotBeforeUpdate()` method is invoked just before the DOM is being rendered. It is used to store the previous values of the state after the DOM is updated.

Any value returned by `getSnapshotBeforeUpdate()` method will be used as a parameter for `componentDidUpdate()` method. This function is always used along with the `componentDidUpdate()` method but vice-versa isn't true.

Syntax:

`getSnapshotBeforeUpdate(prevProps, prevState)`

Parameters: It accepts two parameters, they are *prevProps* and *prevState* which are just the props or state before the component in question is re-rendered.

3.2.2.4 `render`

The `render()` method is of course called when a component gets updated, it has to re-render the HTML to the DOM, with the new changes.

Imagine an example has a button that changes the favorite color to blue.

3.2.2.5 componentDidUpdate

The componentDidUpdate method is called after the component is updated in the DOM.

When the component is mounting it is rendered with the favourite color "red".

When the component has been mounted, a timer changes the state, and the color becomes "yellow".

This action triggers the update phase, and since this component has a componentDidUpdate method.

3.2.3 Unmounting

The next phase in the lifecycle is when a component is removed from the DOM, or unmounting as React likes to call it.

React has only one built-in method that gets called when a component is unmounted:

`componentWillUnmount()`

The componentWillUnmount method is called when the component is about to be removed from the DOM.

3.3 Hooks

Hooks are the new feature introduced in the React 16.8 version. It allows you to use state and other React features without writing a class. Hooks are the functions that "hook into" React state and lifecycle features from function components. It does not work inside classes.

Rules of Hooks

Hooks are similar to JavaScript functions, but you need to follow these two rules when using them. The hooks rule ensures that all the stateful logic in a component is visible in its source code.

These rules are:

1. Only call Hooks at the top level

Do not call Hooks inside loops, conditions, or nested functions. Hooks should always be used at the top level of the React functions. This rule ensures that Hooks are called in the same order each time a component renders.

2. Only call Hooks from React functions

You cannot call Hooks from regular JavaScript functions. Instead, you can call Hooks from React function components. Hooks can also be called from custom Hooks.

Pre-requisites for React Hooks

Node version 6 or above

NPM version 5.2 or above

Create-react-app tool for running the React App

3.3.1 Hooks useState()

Hook state is the new way of declaring a state in React app. Hook uses useState() functional component for setting and retrieving state. Let us understand Hook state with the following example.

App.js

```
import React, { useState } from 'react';  
function CountApp() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>You clicked {count} times</p>  
      <button onClick={() => setCount(count + 1)}>  
        click me  
      </button>  
    </div>  
  );  
}  
export default CountApp;
```

OUTPUT

You clicked 3 times

3.3.2 Hooks useEffect()

The Effect Hook allows us to perform side effects (an action) in the function components. It does not use components lifecycle methods which are available in class components. In other words, Effects Hooks are equivalent to componentDidMount(), componentDidUpdate(), and componentWillUnmount() lifecycle methods.

Side effects have common features that the most web applications need to perform, such as:

Updating the DOM,

Fetching and consuming data from a server API,

Setting up a subscription, etc.

App.js

```
import React, { useState, useEffect } from 'react';

function CounterExample() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

export default CounterExample;

OUTPUT

You clicked 2 times

3.4 Forms

Just like in HTML, React uses forms to allow users to interact with the web page.

Handling Forms

- Handling forms is about how you handle the data when it changes value or gets submitted.
- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components.
- When the components handle the data, all the data is stored in the component state.
- You can control changes by adding event handlers in the onChange attribute.
- We can use the useState() Hook to keep track of each input value and provide a "single source of truth" for the entire application.

Example

SimpleForm.js

```

import React, { useState } from 'react';
const SimpleForm = () => {
  // State variables to store form input values
  const [textBoxValue, setTextBoxValue] = useState('');
  const [textAreaValue, setTextAreaValue] = useState('');
  const [radioValue, setRadioValue] = useState('option1');
  const [checkBoxValue, setCheckBoxValue] = useState(false);
  const [checkBoxValue1, setCheckBoxValue1] = useState(false);
  const [checkBoxValue2, setCheckBoxValue2] = useState(false);
  // Function to handle form submission
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Form submitted!');
    console.log('Text Box Value:', textBoxValue);
    console.log('Text Area Value:', textAreaValue);
    console.log('Radio Value:', radioValue);
    console.log('Check Box Value:', checkBoxValue);
    console.log('Check Box Value:', checkBoxValue1);
    console.log('Check Box Value:', checkBoxValue2);
  };
  return (
    <div>
      <form onSubmit={handleSubmit}>
        {/* Text Box */}
        <label>
          Text Box:
          <input
            type="text"
            value={textBoxValue}
            onChange={(e) => setTextBoxValue(e.target.value)} />
        </label>
        <br />
        {/* Editable Text Area */}
        <label>
          Text Area:
          <textarea
            value={textAreaValue}
            onChange={(e) => setTextAreaValue(e.target.value)} />

```

```

</label>
<br />
{/* Radio Button */}
<div>
  <label>
    <input
      type="radio"
      value="option1"
      checked={radioValue === 'option1'}
      onChange={() => setRadioValue('option1')} />
    Option 1
  </label>
  <label>
    <input type="radio" value="option2" checked={radioValue === 'option2'}
      onChange={() => setRadioValue('option2')} />
    Option 2
  </label>
</div>
<br />
<label>
  <input
    type="checkbox"
    checked={checkBoxValue}
    onChange={() => setCheckBoxValue(!checkBoxValue)}
  />
  Check me
</label>
<label>
  <input
    type="checkbox"
    checked={checkBoxValue1}
    onChange={() => setCheckBoxValue1(!checkBoxValue1)}
  />
  Check me1
</label>
<label>
  <input
    type="checkbox"
    checked={checkBoxValue2}
    onChange={() => setCheckBoxValue2(!checkBoxValue2)}

```

```

    />
    Check me2
  </label>
  <br/>
  { /* Submit Button */ }
  <button type="submit">Submit</button>
</form>
</div>);}};
export default SimpleForm;

```

Output:

Text Box:

Text Area:

☒ Option 1 ☐ Option 2

☐ Check me ☐ Check me1 ☒ Check me2

3.5 Pulling Data from an API

API: API is an abbreviation for Application Programming Interface which is a collection of communication protocols and subroutines used by various programs to communicate between them. A programmer can make use of various API tools to make their program easier and simpler. Also, an API facilitates the programmers with an efficient way to develop their software programs.

Approach: we will know how we fetch the data from API (Application Programming Interface). For the data, we have used the API endpoint from <http://jsonplaceholder.typicode.com/users> we have created the component in App.js and styling the component in App.css. From the API we have target “id”, “name”, “username”, “email” and fetch the data from API endpoints. Below is the stepwise implementation of how we fetch the data from an API in react. We will use the fetch function to get the data from the API.

Step-by-step implementation to fetch data from an api in react:

Step 1: Create React Project

```
npm create-react-app MY-APP
```

Step 2: Change your directory and enter your main folder charting as

```
cd MY-APP
```

Step 3: API endpoint

<https://jsonplaceholder.typicode.com/users>

Step 4: Write code in App.js to fetch data from API and we are using fetch function.

Example:

```
import React from "react";
import './App.css';
class App extends React.Component {
  // Constructor
  constructor(props) {
    super(props);
    this.state = {
      items: [],
      DataisLoaded: false
    };
  }
  // ComponentDidMount is used to execute the code
  componentDidMount() {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((json) => {
        this.setState({
          items: json,
          DataisLoaded: true
        });
      })
  }
  render() {
    const { DataisLoaded, items } = this.state;
    if (!DataisLoaded) return
    <div>
      <h1> Please wait some time.... </h1> </div> ;
    return (
```

EMERGING TRENDS AND TECHNOLOGIES

```
<div className = "App">
  <h1> Fetch data from an api in react </h1>
  {
    items.map((item) => (
      <ol key = { item.id } >
        User_Name: { item.username },
        Full_Name: { item.name },
        User_Email: { item.email }
      </ol>
    )
  )
}
</div>

);
}
}
export default App;
```

Output:

Fetch data from an API in react

User_Name: Bret, Full_Name: Leanne Graham, User_Email: Sincere@april.biz

User_Name: Antonette, Full_Name: Ervin Howell, User_Email: Shanna@melissa.tv

User_Name: Samantha, Full_Name: Clementine Bauch, User_Email: Nathan@yesenia.net

User_Name: Karianne, Full_Name: Patricia Lebsack,
User_Email: Julianne.OConner@kory.org

User_Name: Kamren, Full_Name: Chelsey Dietrich,
User_Email: Lucio_Hettinger@annie.ca

User_Name: Leopoldo_Corkery, Full_Name: Mrs. Dennis Schulist,
User_Email: Karley_Dach@jasper.info

User_Name: Elwyn.Skiles, Full_Name: Kurtis Weissnat, User_Email: Telly.Hoeger@billy.biz

EMERGING TRENDS AND TECHNOLOGIES

User_Name: Maxime_Nienow, Full_Name: Nicholas Runolfsson, V,
User_Email: Sherwood@rosamond.me

User_Name: Delphine, Full_Name: Glenna Reichert,
User_Email: Chaim_McDermott@dana.io

User_Name: Moriah.Stanton, Full_Name: Clementina DuBuque,
User_Email: Rey.Padberg@karina.biz